



F107 – Foundations of Computer Science

Lecture 6

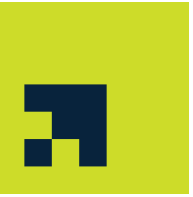
Dr. Sanjit Kumar Saha

Topics



- Programming Languages
 - Types of Programming Languages
 - Types of Execution
 - Compiler
 - Interpreter
 - Programming Paradigms
 - Programming Foundations
 - Pseudocode
- Software Engineering
 - Workflow
 - Tools
- Book Chapters 9, 10, and Appendix C

EVOLUTION



- A computer language is a set of predefined words that are combined into a program according to predefined rules (*syntax*).
- Computer languages have evolved from
 - *machine language,*
 - *assembly language, and*
 - *high-level languages*

Machine languages



- Each computer had its own machine language, which was made of streams of 0s and 1s.
- The only language understood by a computer is machine language.



Table 9.1 Code in machine language to add two integers

<i>Hexadecimal</i>	<i>Code in machine language</i>			
$(1FEF)_{16}$	0001	1111	1110	1111
$(240F)_{16}$	0010	0100	0000	1111
$(1FEF)_{16}$	0001	1111	1110	1111
$(241F)_{16}$	0010	0100	0001	1111
$(1040)_{16}$	0001	0000	0100	0000
$(1141)_{16}$	0001	0001	0100	0001
$(3201)_{16}$	0011	0010	0000	0001
$(2422)_{16}$	0010	0100	0010	0010
$(1F42)_{16}$	0001	1111	0100	0010
$(2FFF)_{16}$	0010	1111	1111	1111
$(0000)_{16}$	0000	0000	0000	0000

Assembly languages



- The idea of replacing binary code for instruction and addresses with symbols or mnemonics.
- These languages were first known as symbolic languages.



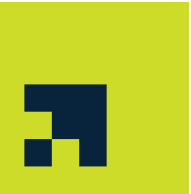
Table 9.2 Code in assembly language to add two integers

<i>Code in assembly language</i>	<i>Description</i>
LOAD RF Keyboard	Load from keyboard controller to register F
STORE Number1 RF	Store register F into Number1
LOAD RF Keyboard	Load from keyboard controller to register F
STORE Number2 RF	Store register F into Number2
LOAD R0 Number1	Load Number1 into register 0
LOAD R1 Number2	Load Number2 into register 1
ADDI R2 R0 R1	Add registers 0 and 1 with result in register 2
STORE Result R2	Store register 2 into Result
LOAD RF Result	Load Result into register F
STORE Monitor RF	Store register F into monitor controller
HALT	Stop

High-level languages



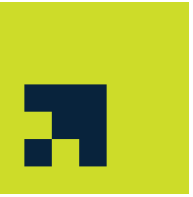
- Although assembly languages greatly improved programming efficiency, they still required programmers to concentrate on the hardware they were using.
- Working with symbolic languages was also very tedious, because each machine instruction had to be individually coded.
- Over the years, various languages, most notably BASIC, COBOL, Pascal, Ada, C, C++ and Java, were developed.



Program 9.1 Addition program in C++

```
/* This program reads two integers from keyboard and prints their sum.
   Written by:
   Date:
*/
#include <iostream.h>
using namespace std;
int main (void)
{
    // Local Declarations
    int number1;
    int number2;
    int result;
    // Statements
    cin >> number1;
    cin >> number2;
    result = number1 + number2;
    cout << result;
    return 0;
} // main
```

TRANSLATION



- Programs today are normally written in one of the high-level languages.
- To run the program on a computer, the program needs to be translated into the machine language of the computer on which it will run.
- The program in a high-level language is called the source program.
- The translated program in machine language is called the object program.
- Two methods are used for translation:
 - **compilation**
 - **interpretation**

Compilation

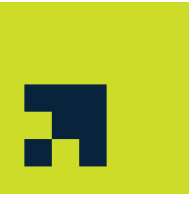


- A compiler normally translates the whole **source program** into the **object program**.

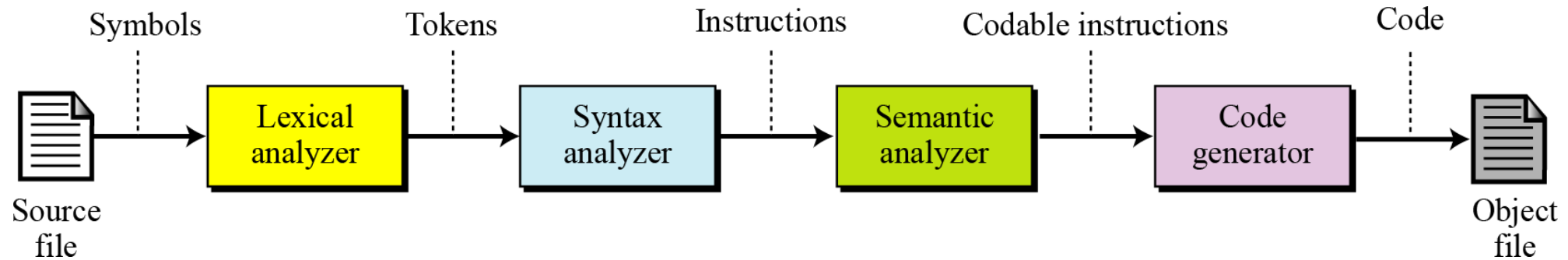
Interpretation

- The process of translating each line of the source program into the corresponding line of the object program and executing the line.

Translation process



- Compilation and interpretation differ in that the first translates the whole source code before executing it, while the second translates and executes the source code a line at a time.
- Both methods, however, follow the same translation process.

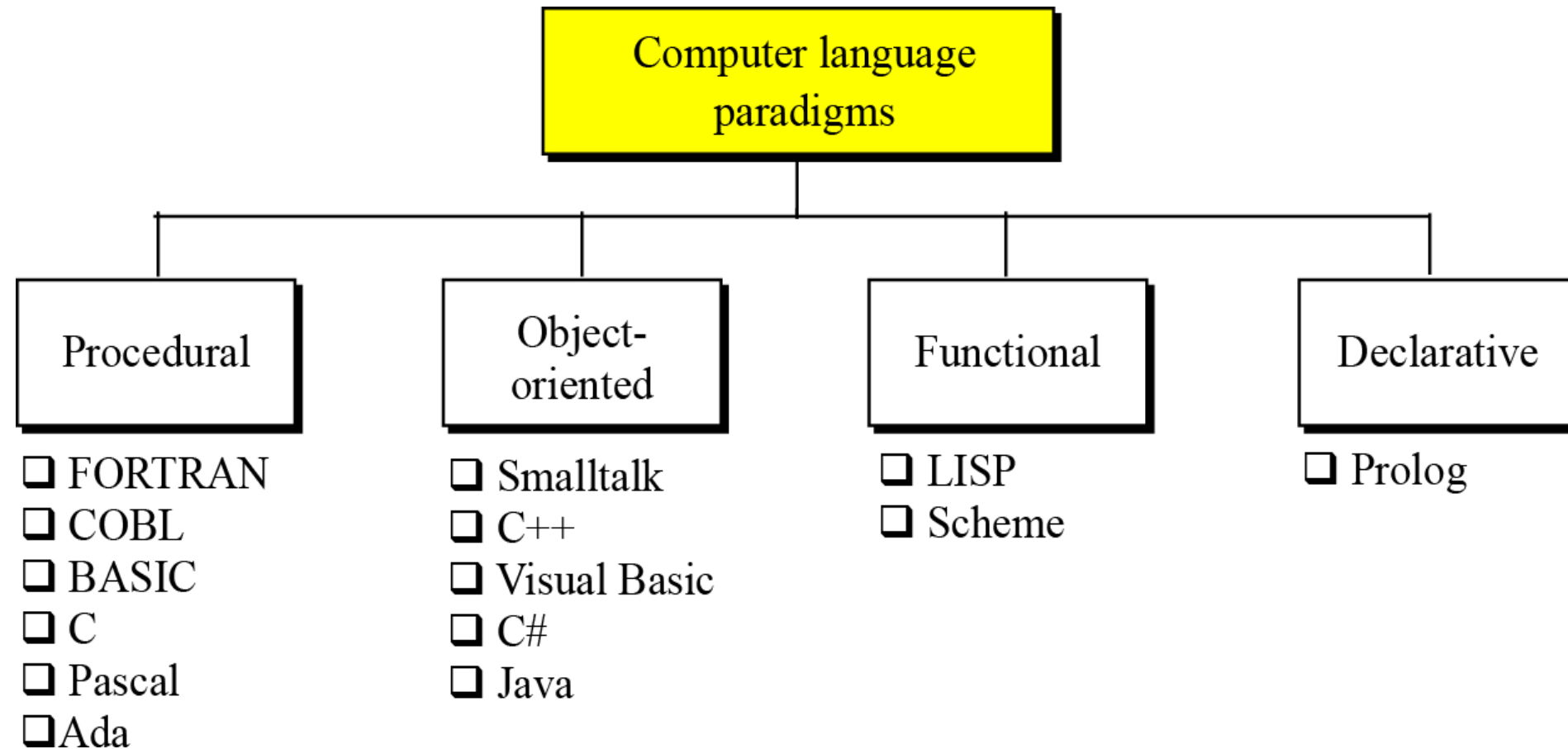


Source code translation process

PROGRAMMING PARADIGMS



- A **paradigm** is a way in which a computer language looks at the problem to be solved.
- We divide computer languages into four paradigms:
 - *procedural*
 - *object-oriented*
 - *functional*
 - *declarative*

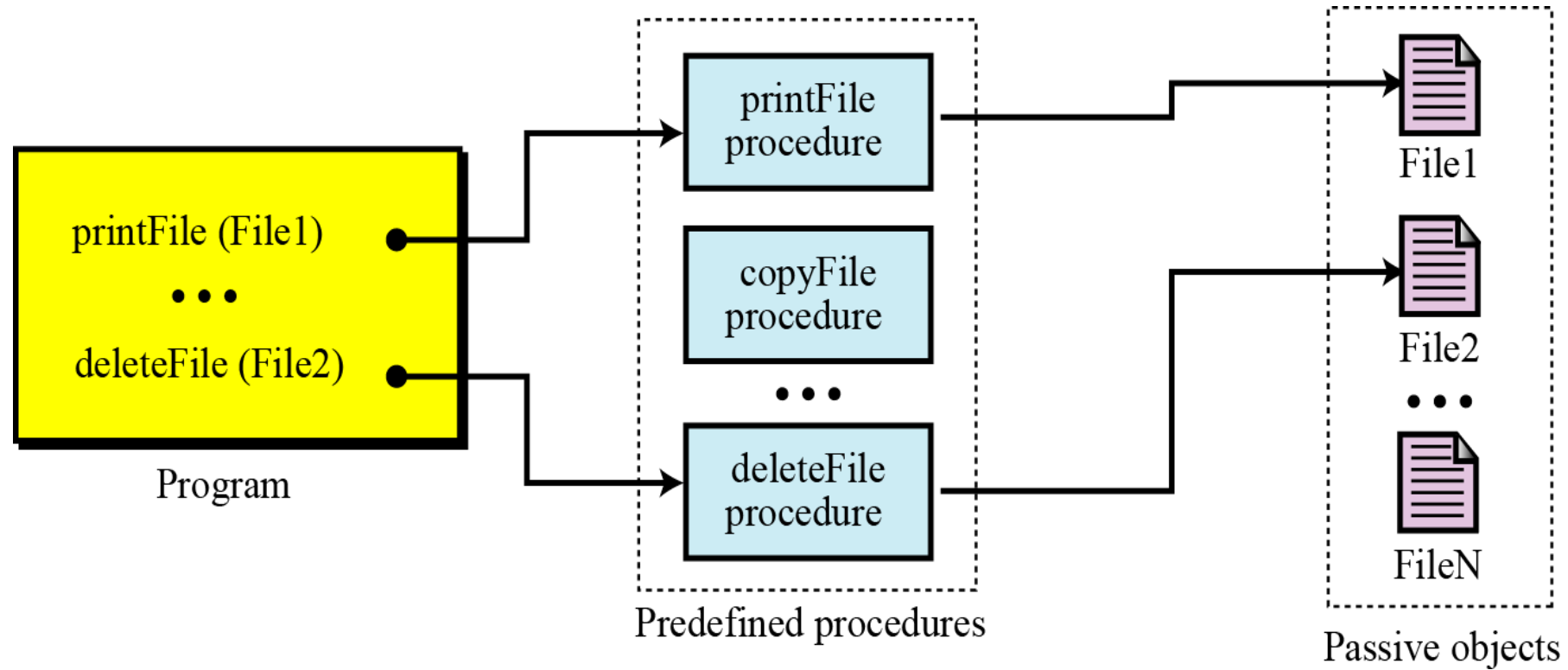
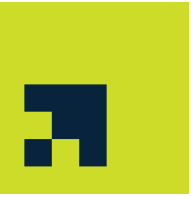


Categories of programming languages

The procedural paradigm

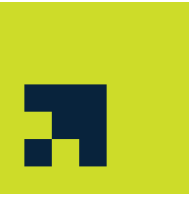


- A program in a procedural paradigm is an active agent that uses passive objects that we refer to as data or data items.
- To manipulate a piece of data, the active agent (program) issues an action, referred to as a **procedure**.
- For example, think of a program that prints the contents of a file.
 - The file is a passive object.
 - To print the file, the program uses a procedure, which we call print.



The concept of the procedural paradigm

Some procedural languages

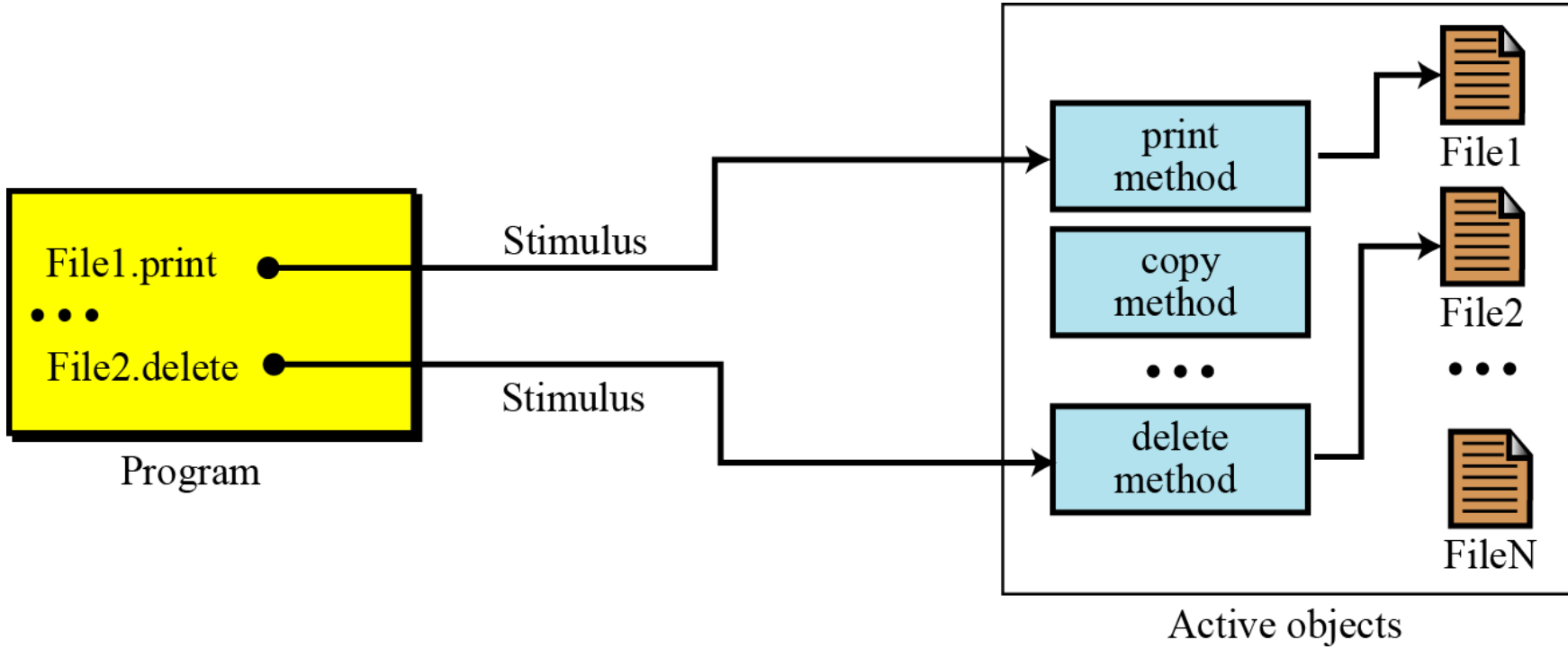
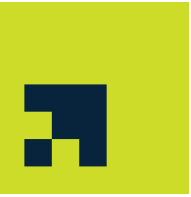


- FORTRAN (FORmula TRANslation)
- COBOL (COmmon Business-Oriented Language)
- Pascal
- C
- Ada

The object-oriented paradigm



- The object-oriented paradigm deals with active objects instead of passive objects.
- We encounter many active objects in our daily life: a vehicle, an automatic door, a dishwasher and so on.
- The action to be performed on these objects are included in the object: the objects need only to receive the appropriate stimulus from outside to perform one of the actions.



The concept of an object-oriented paradigm

Some object-oriented languages



- C++
- Java

The functional paradigm

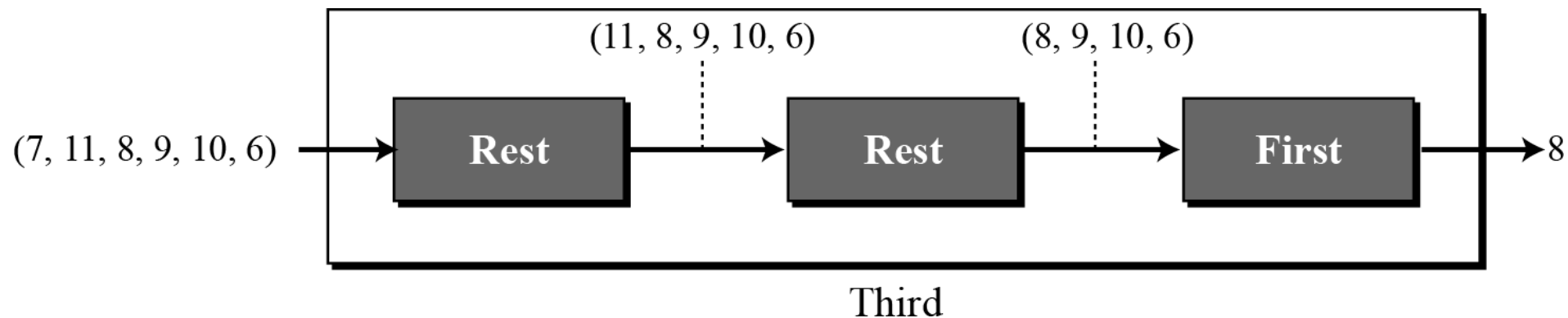


- In the functional paradigm a program is considered a mathematical function.
- In this context, a function is a black box that maps a list of inputs to a list of outputs.



A function in a functional language

For example, we can define a primitive function called `first` that extracts the first element of a list. It may also have a function called `rest` that extracts all the elements except the first. A program can define a function that extracts the third element of a list by combining these two functions.



Extracting the third element of a list

Some functional languages



- LISP (LISt Programming)
- Scheme

The declarative paradigm



- A declarative paradigm uses the principle of logical reasoning to answer queries.
- It is based on formal logic defined by Greek mathematicians and later developed into first-order predicate calculus.
- Logical reasoning is based on deduction.
- Some statements (facts) are given that are assumed to be true, and the logician uses solid rules of logical reasoning to deduce new statements (facts).
- For example, the famous rule of deduction in logic is:

If (A is B) and (B is C), then (A is C)



Using this rule and the two following facts,

Fact 1: Socrates is a human $\rightarrow$ A is B
Fact 2: A human is mortal $\rightarrow$ B is C

we can deduce a new fact:

Fact 3: Socrates is mortal $\rightarrow$ A is C

Prolog



- One of the famous declarative languages is Prolog (PROgramming in LOGic), developed by A. Colmerauer in France in 1972.
- A program in Prolog is made up of facts and rules.
- For example, the previous facts about human beings can be stated as:

```
human (John)  
mortal (human)
```

The user can then ask:

```
?-mortal (John)
```

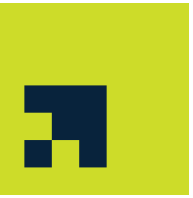
and the program will respond with *yes*.

COMMON CONCEPTS



- We conduct a quick navigation through some procedural languages to find common concepts.
- Some of these concepts are also available in most object-oriented languages.
 - An object-oriented paradigm uses the procedural paradigm when creating methods.

Programming Foundations

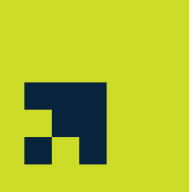


Examples use C

- Identifiers
 - Names/symbols for programming entities
 - Some names are reserved
- Data types
 - Defines the range of values it can hold and the possible operations
 - Often also implies the way it is stored, as well as its storage size
 - Simple types (scalars)
 - `boolean`, `char`, `string`, `int`, `float`, ...
 - Composite types (records)
 - `array`, `list`, `map`, ...
 - `struct`, `union`, `object`, ...

```
// example
struct address {
    char *name;
    bool isLocal;
    int zipCode;
};
```

Programming Foundations



- Variables

- Modifiable memory locations
- Name used instead of address of the location
- Declaration and initialization

```
int number; // careful!  
  
int initNumber = 0;
```

- Literals

- Specify certain values in code (often data type is just implied) ($\pi = 3.14$)
- Often used in expressions or when initializing variables

- Constants

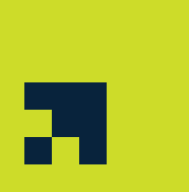
- Specific construct to declare a certain unchangeable variable

```
const int value = 19;
```

- Comments

- Descriptions that enhance understanding of code

Programming Foundations



- Expressions

- Sequence of operands and operators

```
3 + (4 * 5) >> 2
```

- Operators

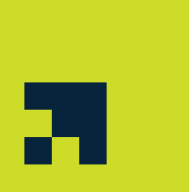
- Arithmetic operators: `+, -, *, /, %, ++, --`
- Shift operators: `>>, <<`
- Relational operators: `>, >=, <, <=, ==, !=`
- Logical operators: `!, &, &&, |, ||, ^`

- Statements

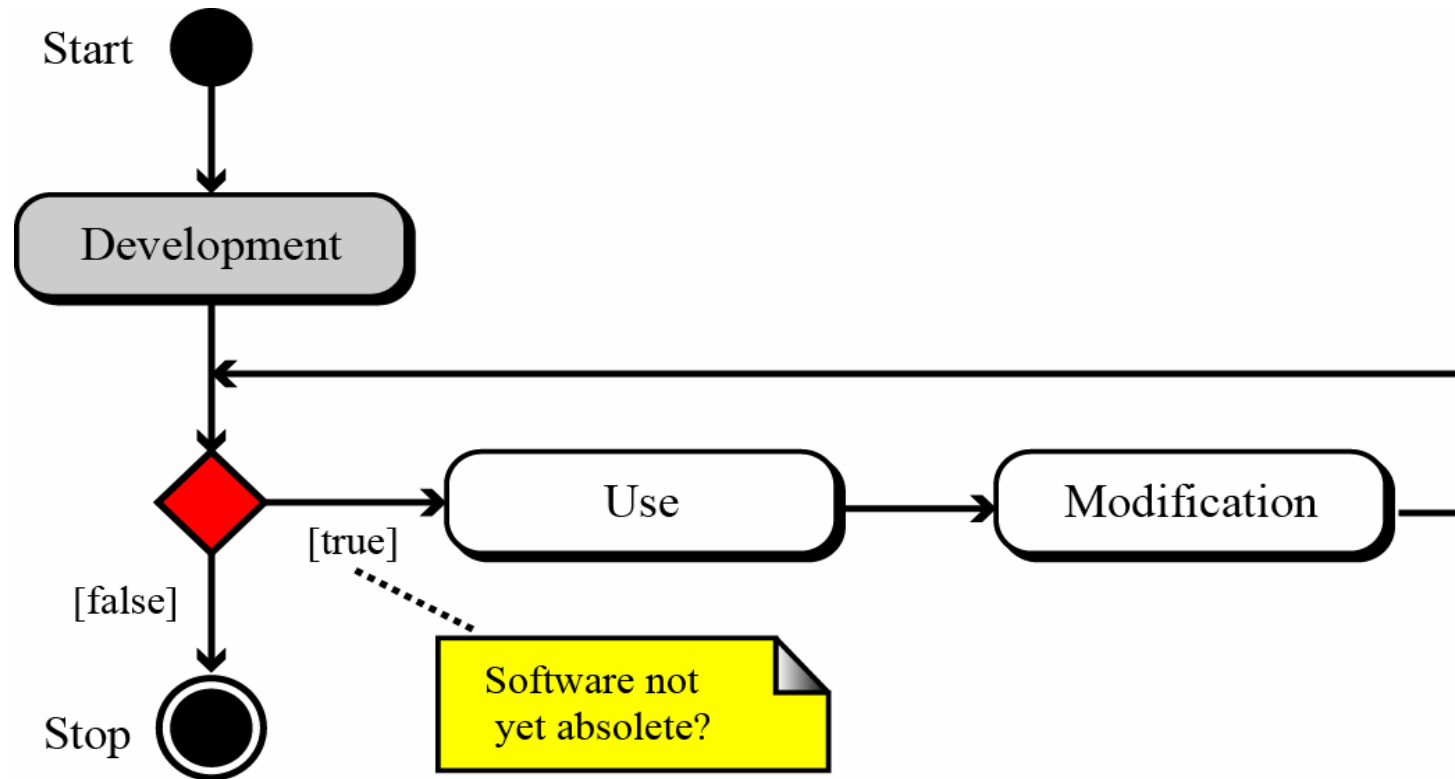
- Performs an action
 - Executes function (w/ or w/o return value)
 - Assigns value

```
y = myFunc(x);  
otherFunc();
```

THE SOFTWARE LIFECYCLE

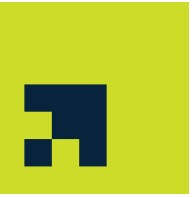


- A fundamental concept in **software engineering** is the *software lifecycle*.
- Software, like many other products, goes through a cycle of repeating phases.



The software lifecycle

Development process models



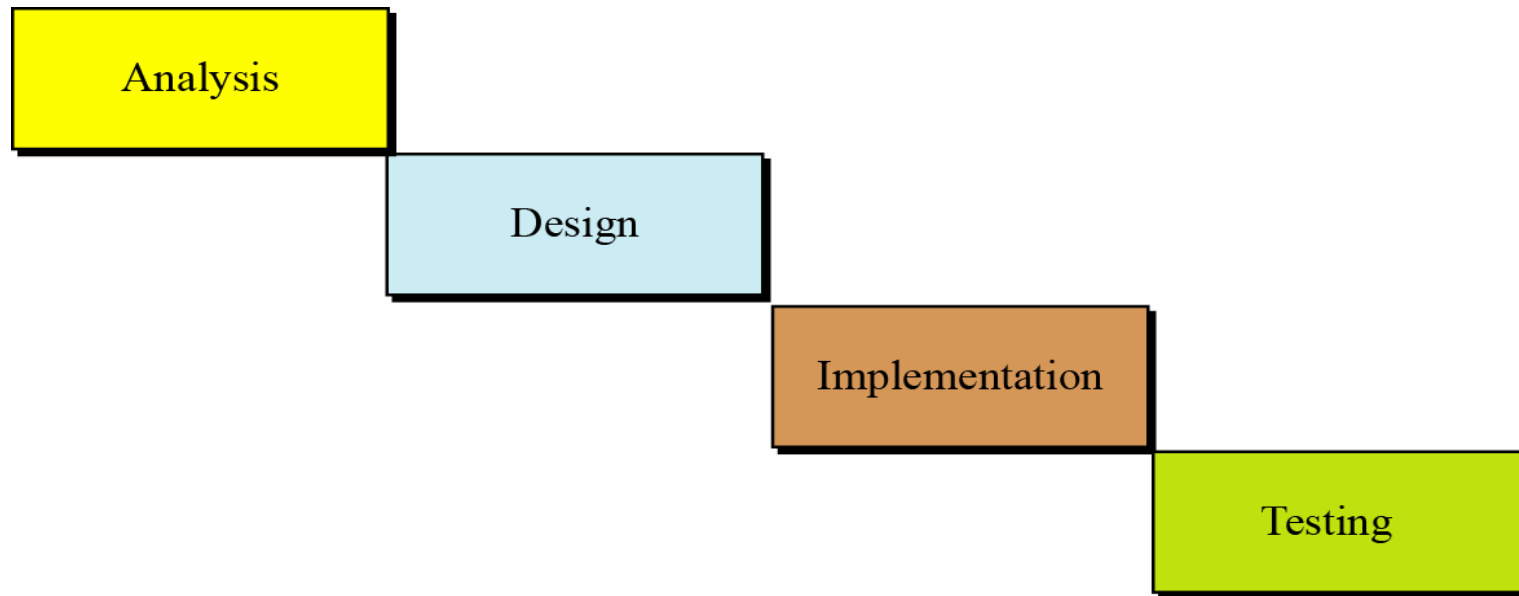
- The development process in the software lifecycle involves four phases:
 - analysis,
 - design,
 - implementation and
 - testing
- There are several models for the development process.
- Two most common here:
 - the waterfall model
 - the incremental model

The waterfall model



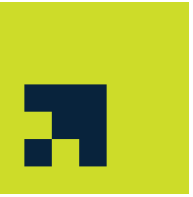
In this model, the development process flows in only one direction.

This means that a phase cannot be started until the previous phase is completed.

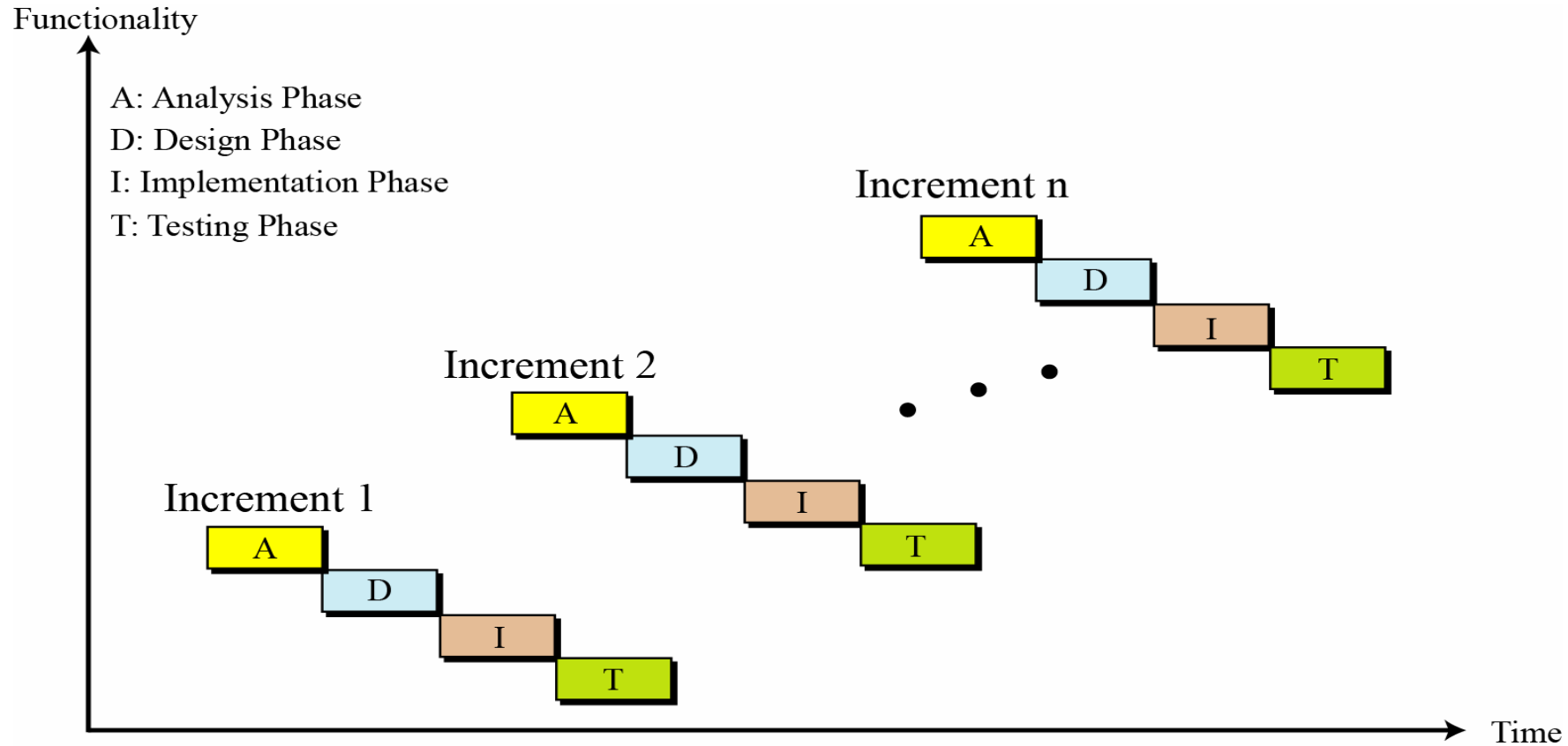


The waterfall model

The incremental model

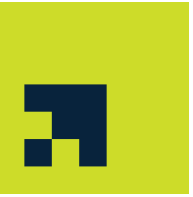


In the incremental model, software is developed in a series of steps.



The incremental model

ANALYSIS PHASE



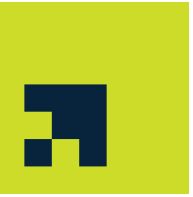
- The development process starts with the **analysis phase**.
- This phase results in a specification document that shows what the software will do without specifying how it will be done.
- The analysis phase can use two separate approaches, depending on whether the implementation phase is done using
 - a procedural programming language or
 - an object-oriented language

Procedure-oriented analysis



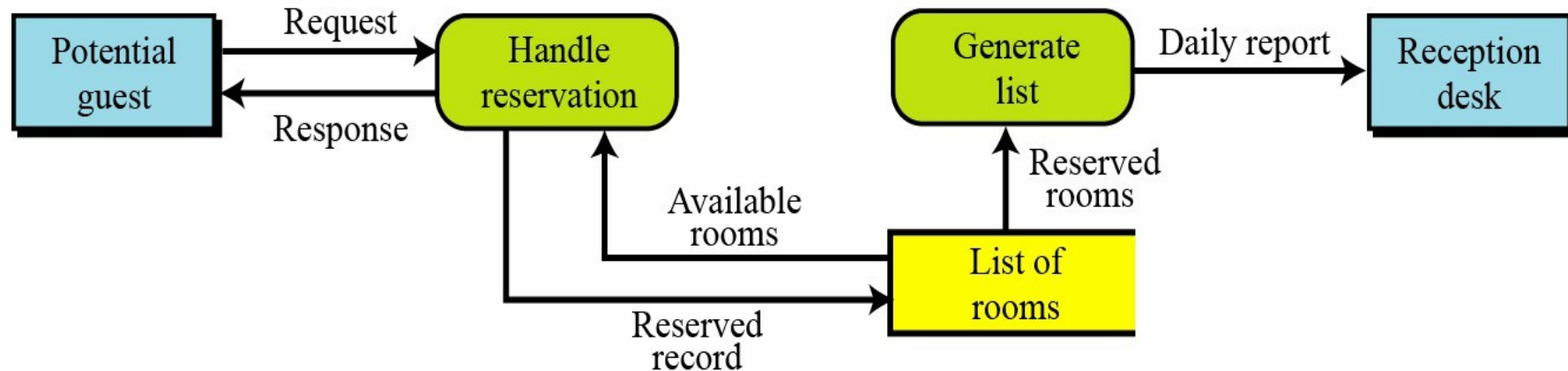
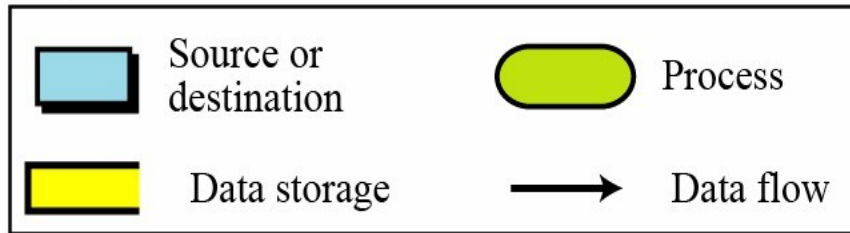
- Procedure-oriented analysis—also called structured analysis or classical analysis—is the analysis process used if the system implementation phase will use a procedural language.
- The specification in this case may use several modeling tools, but we discuss only a few of them here.

Data flow diagrams



Data flow diagrams show the movement of data in the system.

Legend



An example of a data flow diagram

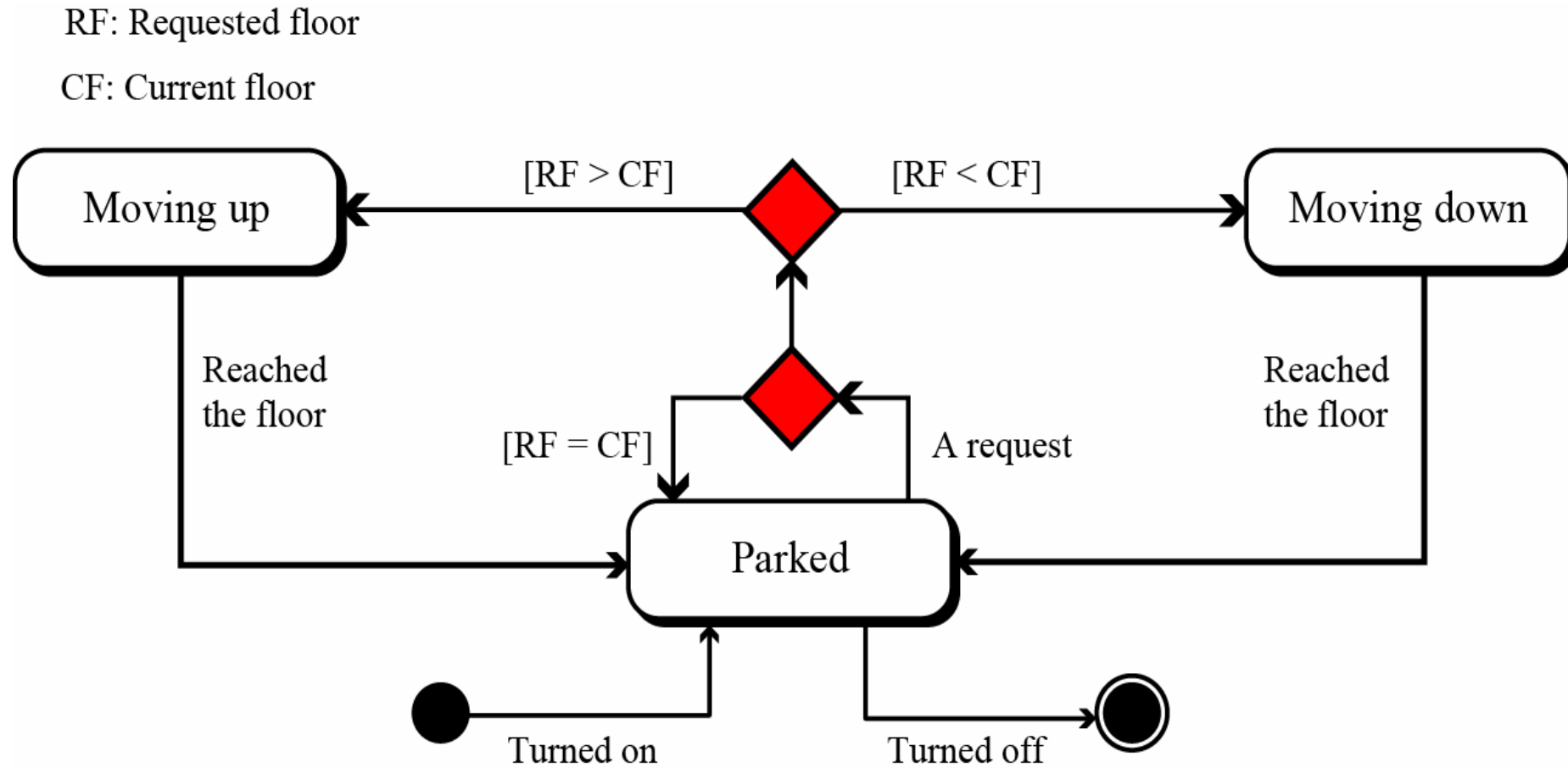
Entity-relationship diagrams



- Another modeling tool used during the analysis phase is the entity-relationship diagram.
- This diagram is also used in database design.

State diagrams

- State diagrams provide another useful tool that is normally used when the state of the entities in the system will change in response to events.
- For example, we show the operation of a one-passenger elevator. When a floor button is pushed, the elevator moves in the requested direction. It does not respond to any other request until it reaches its destination.



An example of a state diagram

Object-oriented analysis

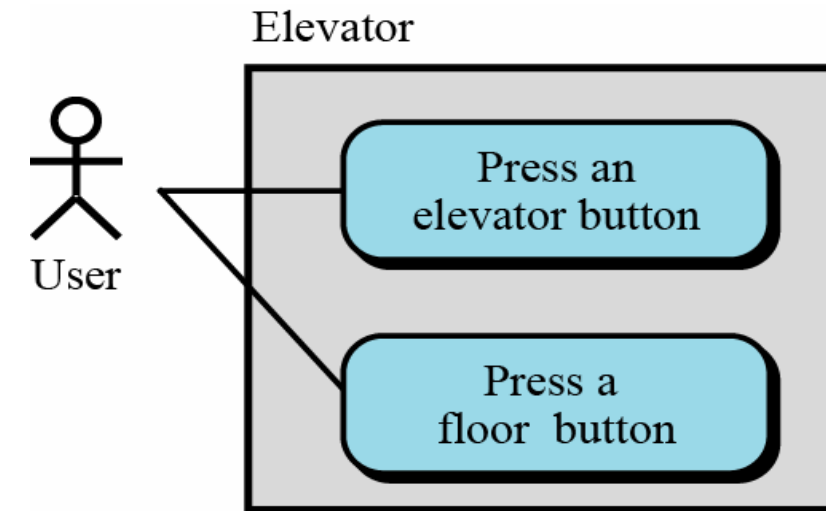


- Object-oriented analysis is the analysis process used if the implementation uses an object-oriented language.
- The specification document in this case may use several tools, but we discuss only a few of them here.

Use case diagrams



- A use-case diagram gives the user's view of a system:
 - it shows how users communicate with the system.
- A use-case diagram uses four components:
 - *system*
 - *use cases*
 - *actors*
 - *relationships*
- A system, shown by a rectangle, performs a function.

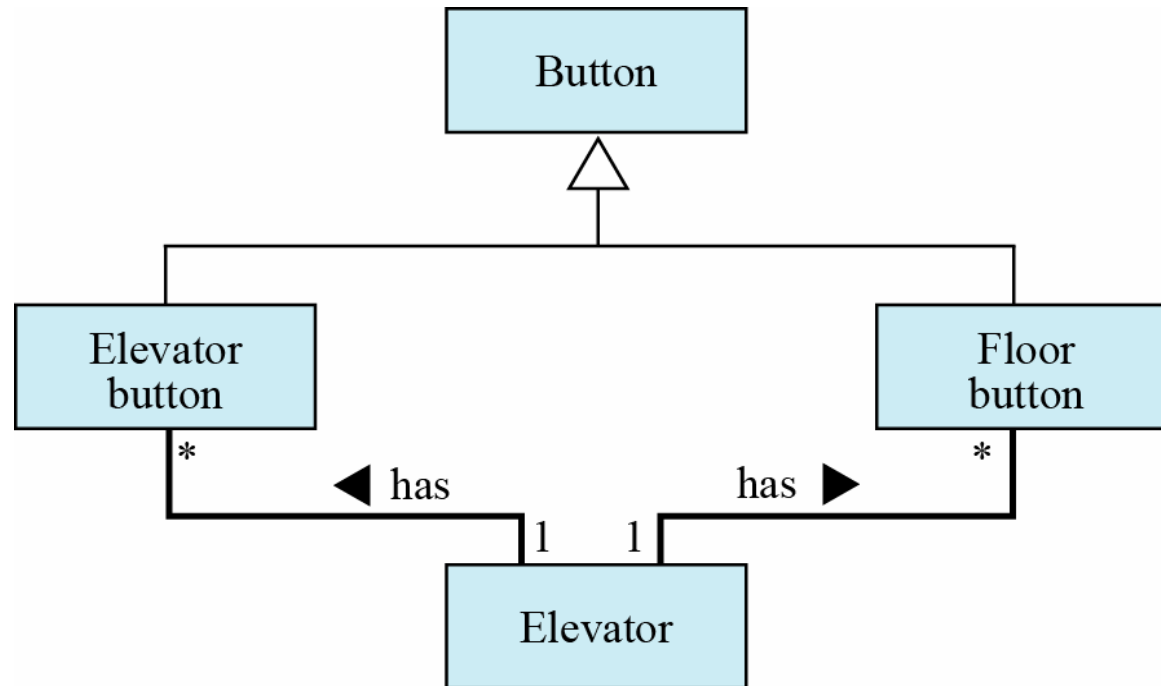


Use case diagram

Class diagrams

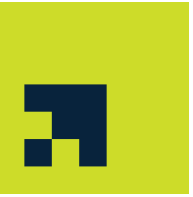


- The next step in analysis is to create a class diagram for the system.
- For example, we can create a class diagram for our old-style elevator.
 - To do so, we need to think about the entities involved in the system.



An example of a class diagram

State chart



- After the class diagram is finalized, a state chart can be prepared for each class in the class diagram.
- A state chart in object-oriented analysis plays the same role as the state diagram in procedure-oriented analysis.
- This means that for the class diagram, we need to have a four-state chart.

DESIGN PHASE



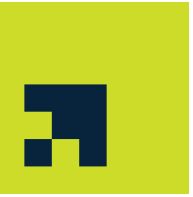
- The design phase defines how the system will accomplish what was defined in the analysis phase.
- In the design phase, all components of the system are defined.

Procedure-oriented design

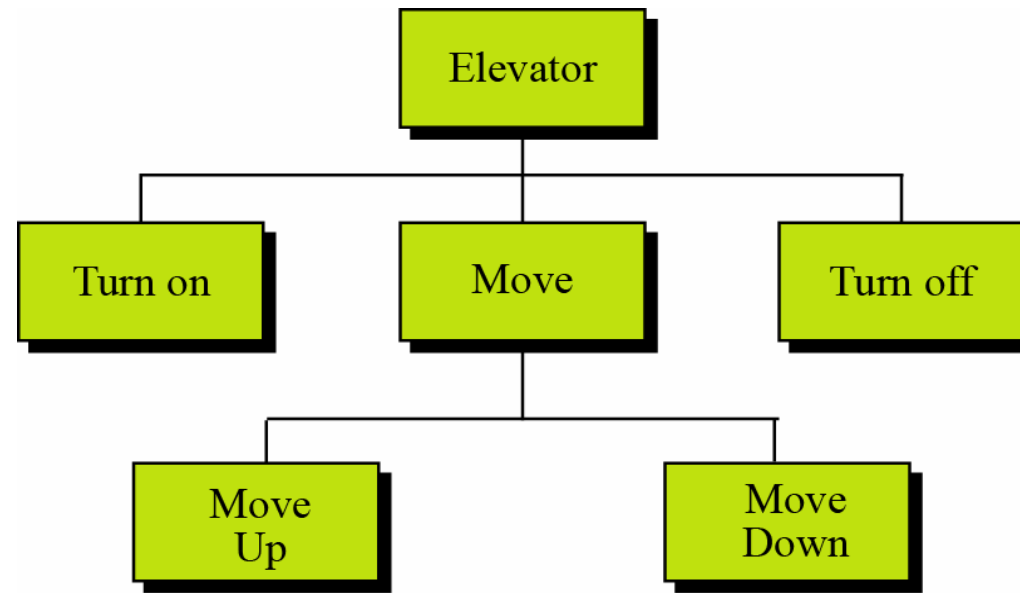


- In **procedure-oriented design** we have both procedures and data to design.
- We discuss a category of design methods that concentrate on procedures.
- In procedure-oriented design, the whole system is divided into a set of procedures or modules.

Structure charts



- A common tool for illustrating the relations between modules in procedure-oriented design is a structure chart.
- For example, the elevator system whose state diagram can be designed as a set of modules shown in the structure chart.



A structure chart

Modularity



- Modularity means breaking a large project into smaller parts that can be understood and handled easily.
- In other words, modularity means dividing a large task into small tasks that can communicate with each other.
- Two main concerns when a system is divided into modules: **coupling** and **cohesion**.
 - Coupling is a measure of how tightly two modules are bound to each other.

Coupling between modules in a software system must be minimized

- Cohesion is a measure of how closely the modules in a system are related.

Cohesion between modules in a software system must be maximized

Object-oriented design



- In object-oriented design the design phase continues by elaborating the details of classes.
- A class is made of a set of variables (attributes) and a set of methods.
- The object-oriented design phase lists details of these attributes and methods.

Button	Floor button	Elevator button	Elevator
status: (on, off)			
turnOn turnOff	turnOn turnOff	turnOn turnOff	moveUp moveDown

An example of classes with attributes and methods

IMPLEMENTATION PHASE



- In the waterfall model, after the design phase is completed, the implementation phase can start.
- In this phase the programmers write the code for the modules in procedure-oriented design, or write the program units to implement classes in object-oriented design.
- There are several issues we need to mention in each case.

Choice of language



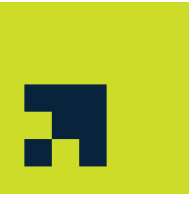
- In a procedure-oriented development, the project team needs to choose a language or a set of languages from among the procedural languages.
- C++ are considered to be both a procedural and an object-oriented language.
- Normally an implementation uses a purely procedural language such as C.

Software quality

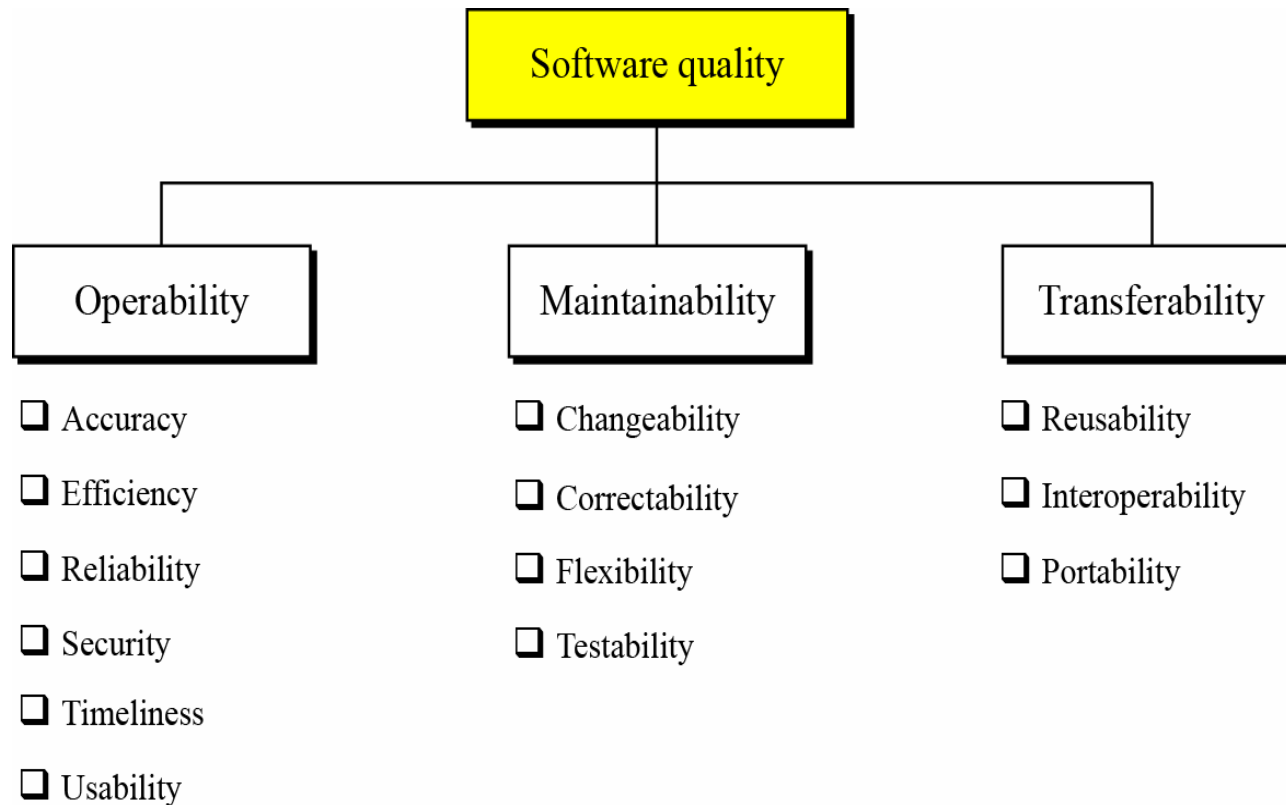


- A software system of high quality is one that
 - satisfies the user's requirements,
 - meets the operating standards of the organization, and
 - runs efficiently on the hardware for which it was developed
- If we want to achieve a software system of high quality, we must be able to define some attributes of quality.

Software quality factors

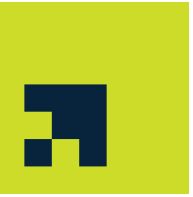


Software quality can be divided into three broad measures: *operability*, *maintainability* and *transferability*.

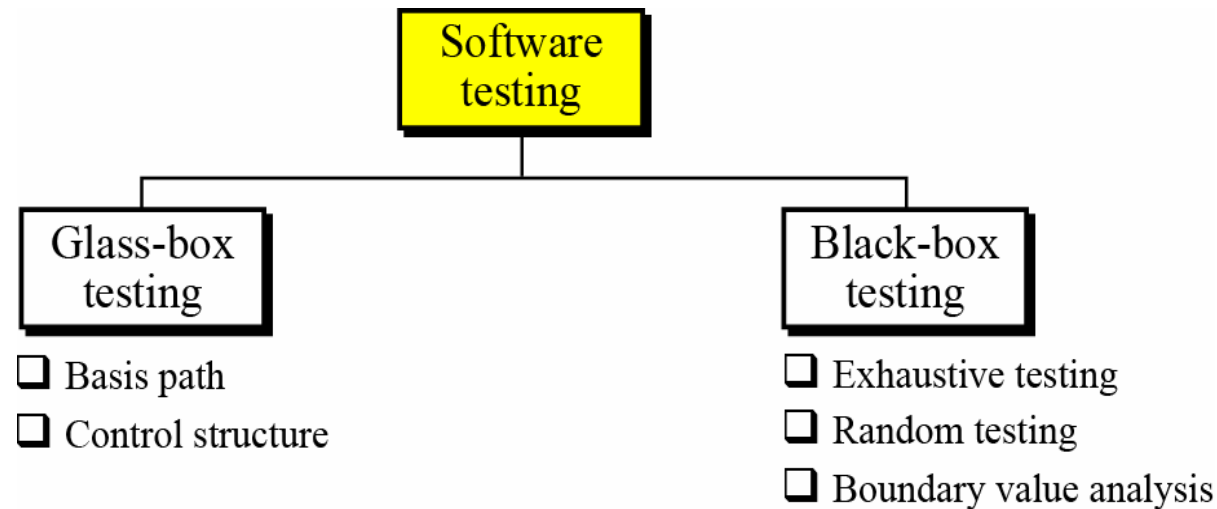


Quality factors

TESTING PHASE

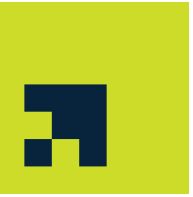


- The goal of the testing phase is to find errors, which means that a good testing strategy is the one that finds most errors.
- There are two types of testing: **glass-box** and **black-box**.



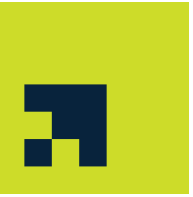
Software testing

Glass-box testing



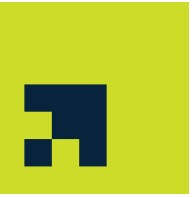
- **Glass-box testing** (or **white-box testing**) is based on knowing the internal structure of the software.
- The testing goal is to determine whether all components of the software do what they are designed for.
- Glass-box testing assumes that the tester knows everything about the software. In this case, the software is like a glass box in which everything inside the box is visible.
- Glass-box testing is done by the software engineer or a dedicated team.

Black-box testing



- Black box testing gets its name from the concept of testing software without knowing what is inside it and without knowing how it works.
- In other words, the software is like a black box into which the tester cannot see.
- Black-box testing tests the functionality of the software in terms of what the software is supposed to accomplish, such as its inputs and outputs.

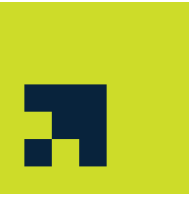
DOCUMENTATION



- For software to be used properly and maintained efficiently, documentation is needed.
- Usually, three separate sets of documentation are prepared for software:
 - **user documentation**
 - **system documentation**
 - **technical documentation**

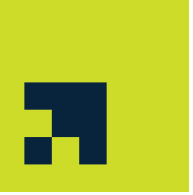
Documentation is an ongoing process.

User documentation



- To run the software system properly, the users need documentation, traditionally called a *user guide*, that shows how to use the software step by step.
- User guides usually contains a tutorial section to guide the user through each feature of the software.
- A good user guide can be a very powerful marketing tool: the importance of user documentation in marketing cannot be over-emphasized.
- User guides should be written for both the novice and the expert users, and a software system with good user documentation will definitely increase sales.

System documentation



- System documentation defines the software itself.
- It should be written so that the software can be maintained and modified by people other than the original developers.
- System documentation should exist for all four phases of system development.

Technical documentation

- Technical documentation describes the installation and the servicing of the software system.
- Installation documentation defines how the software should be installed on each computer, for example, servers and clients.
- Service documentation defines how the system should be maintained and updated if necessary.